File I/O

Read, Write, and Process Files in Python

LEARNING OUTCOME

By the end of this module you will read and write text and binary files, work with CSV and JSON data, use pathlib for cross-platform path handling, and process large files efficiently without loading them into memory.

Skills You Will Gain
• Open files safely with context managers (with statement)
• Read entire files, line by line, or in chunks
• Write and append to text files
• Work with CSV using csv module
• Parse and create JSON data
• Use pathlib for modern path operations

■ **File I/O is essential for every real program. Configuration files, data imports, log files, reports -- all use file operations. Python's context managers make file handling safe and clean.**

SECTION 1

Reading and Writing Files

```
# ALWAYS use context manager -- auto-closes file even on error with open("data.txt", "r", encoding="utf-8") as f: content = f.read() # read entire file as string with open("data.txt", "r", encoding="utf-8") as f: lines = f.readlines() # list of lines (includes \n) with open("data.txt", "r", encoding="utf-8") as f: for line in f: # BEST: iterate line by line (memory efficient) line = line.strip() # remove trailing \n and whitespace if line: # skip empty lines process(line) # File open modes # "r" read (default) # "w" write (creates new or OVERWRITES existing) # "a" append (adds to end) # "x" exclusive create (fails if file exists) # "rb" read binary # "wb" write binary # "r+" read and write # WRITING lines = ["Line 1", "Line 2", "Line 3"] with open("output.txt", "w", encoding="utf-8") as f: f.write("First line\n") # write string f.writelines(line + "\n" for line in lines) # write multiple # APPENDING with open("log.txt", "a", encoding="utf-8") as f: f.write(f"[2024-01-15] New log entry\n") # Reading with seek and tell with open("data.txt","r") as f: pos = f.tell() # current position chunk = f.read(1024) # read 1024 bytes f.seek(0) # go back to start f.seek(-100, 2) # 100 bytes before end # Binary files with open("image.png","rb") as f: header = f.read(8) # read first 8 bytes data = f.read() # read rest with open("copy.png","wb") as f: f.write(data)
```

SECTION 2

CSV and JSON

```
import csv import json # CSV READING with open("data.csv","r",newline="",encoding="utf-8") as f: reader = csv.reader(f) header = next(reader) # skip header row for row in reader: name, age, city = row # unpack columns print(f"{name} ({age}) from {city}") # DictReader -- rows as dicts (header row = keys) with open("data.csv","r",newline="",encoding="utf-8") as f: reader = csv.DictReader(f) data = [] for row in reader: data.append({ "name": row["name"], "age": int(row["age"]), "city": row["city"] }) # CSV WRITING records = [ {"name":"Alice","age":28,"city":"Mumbai"}, {"name":"Bob","age":35,"city":"Delhi"}, ] with open("output.csv","w",newline="",encoding="utf-8") as f: writer = csv.DictWriter(f, fieldnames=["name","age","city"]) writer.writeheader() # write header row writer.writerows(records) # write all data rows # JSON READING with open("config.json","r",encoding="utf-8") as f: config = json.load(f) # file -> Python dict json_string = '{"name":"Alice","age":28}' data = json.loads(json_string) # string -> Python dict # JSON WRITING result = {"status":"success","count":42,"data":[1,2,3]} with open("result.json","w",encoding="utf-8") as f: json.dump(result, f, indent=2, ensure_ascii=False) json_str = json.dumps(result, indent=2) # dict -> string # JSON type mapping # Python dict <-> JSON object {} # Python list <-> JSON array [] # Python str <-> JSON string "" # Python int/float <-> JSON number # Python True/False <-> JSON true/false # Python None <-> JSON null
```

SECTION 3

pathlib -- Modern Path Handling

```
from pathlib import Path import shutil # Create path objects home = Path.home() # /home/user cwd = Path.cwd() # current directory fp = Path("data/reports/2024.csv") abs_fp = fp.resolve() # absolute path # Path components print(fp.name) # "2024.csv" print(fp.stem) # "2024" print(fp.suffix) # ".csv" print(fp.parent) # data/reports print(fp.parts) # ("data","reports","2024.csv") # Building paths (OS-independent) data_dir = Path("data") report = data_dir / "reports" / "2024.csv" # / operator! # File operations report.exists() # True/False report.is_file() # True/False report.is_dir() # True/False report.stat().st_size # file size in bytes report.stat().st_mtime # last modified timestamp # Read/write (no need to open!) text = report.read_text(encoding="utf-8") bytes_data = report.read_bytes() report.write_text("new content", encoding="utf-8") # Directory operations Path("new_dir").mkdir(parents=True, exist_ok=True) shutil.copy(src, dst) # copy file shutil.move(src, dst) # move file report.unlink() # delete file shutil.rmtree("old_dir") # delete directory tree # Glob -- find files matching pattern for csv_file in Path("data").glob("**/*.csv"): # recursive print(csv_file) for py_file in Path(".").glob("*.py"): # current dir only
```

```
print(py_file.name) # List directory for item in Path("data").iterdir(): if item.is_file(): print(f"{item.name}: {item.stat().st_size} bytes")
```

■ PRACTICE Q & A

Q1. Why should you always use 'with open()' instead of open() + close()?

A: *The with statement guarantees the file is closed even if an exception occurs. Without it, if an exception happens between open() and close(), the file stays open -- causing resource leaks and potential data corruption.*

Q2. What is the difference between f.read(), f.readline(), and f.readlines()?

A: *read(): reads entire file as one string. readline(): reads one line at a time (call repeatedly). readlines(): reads all lines into a list. For large files, iterate with 'for line in f:' which is memory-efficient.*

Q3. What is the difference between json.load() and json.loads()?

A: *load() reads from a FILE object. loads() parses a STRING. Similarly, json.dump() writes to a file, json.dumps() returns a string. load/dump = file, loads/dumps = string.*

Q4. Why use pathlib instead of os.path?

A: *pathlib provides an object-oriented interface with / operator for joining paths (works cross-platform). Methods like .read_text(), .write_text(), .exists() are cleaner than os.path equivalents. Available Python 3.4+.*

Q5. How do you process a very large file without loading it all into memory?

A: *Iterate line by line: 'for line in f:' reads one line at a time. Or read in chunks: 'while chunk := f.read(8192):'. Never use f.read() or f.readlines() on large files.*

■ File I/O Cheat Sheet	
`with open(path,'r',encoding='utf-8') as f:`	Safe file open
`f.read()`	Read entire file as string
`for line in f:`	Line by line (memory efficient)
`line.strip()`	Remove trailing newline/spaces
`open(path,'w')`	Write (creates or overwrites)
`open(path,'a')`	Append to file
`f.write('text\n')`	Write string to file
`csv.DictReader(f)`	CSV rows as dicts
`csv.DictWriter(f,fieldnames)`	Write dicts to CSV
`json.load(f) / json.loads(s)`	JSON file/string to dict
`json.dump(d,f,indent=2)`	Dict to JSON file
`Path('dir') / 'file.txt'`	Cross-platform path join